A value of type `Free[F,A]` is like a program written in an instruction set provided by `F`. In the case of `Console`, the two instructions are `PrintLine` and `ReadLine`. The recursive scaffolding (`Suspend`) and monadic variable substitution (`FlatMap` and `Return`) are provided by `Free` itself. We can introduce other choices of `F` for different instruction sets, for example, different I/O capabilities—a file system `F` granting read/write access (or even just read access) to the file system. Or we could have a network `F` granting the ability to open network connections and read from them, and so on.

13.4.3 Pure interpreters

Note that nothing about the `ConsoleIO` type implies that any effects must actually occur! That decision is the responsibility of the interpreter. We could choose to translate our `Console` actions into pure values that perform no I/O at all! For example, an interpreter for testing purposes could just ignore `PrintLine` requests and always return a constant string in response to `ReadLine` requests. We do this by translating our `Console` requests to a `String => A`, which forms a monad in `A`, as we saw in chapter 11, exercise 11.20 (`readerMonad`).

Listing 13.7 Creating our `ConsoleReader` class

```
case class ConsoleReader[A](run: String => A) {
  def map[B](f: A => B): ConsoleReader[B] =
    ConsoleReader(r => f(run(r)))
  def flatMap[B](f: A => ConsoleReader[B]): ConsoleReader[B] =
    ConsoleReader(r => f(run(r)).run(r))
}
object ConsoleReader {
  implicit val monad = new Monad[ConsoleReader] {
    def unit[A](a: => A) = ConsoleReader(_ => a)
    def flatMap[A,B](ra: ConsoleReader[A])(f: A => ConsoleReader[B]) =
      ra flatMap f
  }
}
```

A specialized reader monad

We introduce another function on `Console`, `toReader`, and then use that to implement `runConsoleReader`:

```
sealed trait Console[A] {
  ...
  def toReader: ConsoleReader[A]
}
val consoleToReader = new (Console ~> ConsoleReader) {
  def apply[A](a: Console[A]) = a.toReader
}

@annotation.tailrec
def runConsoleReader[A](io: ConsoleIO[A]): ConsoleReader[A] =
  runFree[Console,ConsoleReader,A](io)(consoleToReader)
```

Or for a more complete simulation of console I/O, we could write an interpreter that uses two lists—one to represent the input buffer and another to represent the